Lab Number and Title: Lab 1 - Introduction to HDL using Verilog - Logic Gates

Name: Jumier Rey U. Cabañero

E-mail: jumerrey.cabanero@g.msuiit.edu.ph

Lab Section: M89

1. Lab Purpose

The lab assignment Lab 1 - Introduction to HDL using Verilog - Logic Gates introduces designing, simulating, and verifying fundamental digital logic gates (OR, NOT, NAND, NOR, XOR, and XNOR) using Verilog HDL. It requires utilizing three different modeling styles, Dataflow, Behavioral, and Structural, alongside writing a standalone testbench module for each gate to systematically cycle through all possible input combinations. For simulation and verification, the manual specifies using Icarus Verilog and GTKWave, however, we can alternatively use Intel Quartus Prime and ModelSim as permitted by the instructor.

2. Implementation Details

To implement and verify the fundamental logic gates (OR, NOT, NAND, NOR, XOR and XNOR), I used Intel Quartus Prime alongside ModelSim. I constructed Verilog modules for each gate across three distinct HDL modeling styles, behavioral, dataflow, and structural, along with corresponding testbenches as guided by the lab manual. During development, I noted that behavioral models require declaring outputs as reg types because values are assigned within procedural always blocks. Detailed line-by-line comments were added to every Verilog file to document the underlying logic. To maintain project hierarchy and prevent source files from unlinking upon reopening Quartus, I explicitly registered all .v files for the gate variants and testbenches into the project file list via Project -> Add/Remove Files in Project.

For each gate, the unified testbench instantiated all three modeling styles simultaneously (dataflow, behavioral, and structural) to evaluate their responses. Under Settings > EDA Tool Settings > Simulation, I configured ModelSim-Altera as the simulation tool with Verilog HDL output, and edited the Compile test bench setup to explicitly include all corresponding gate design files alongside the testbench file. This ensured complete compilation scripts without errors. Finally, after completing a full project compilation in Quartus to 100% without errors, I launched the RTL Simulation in ModelSim to inspect the concurrent signal waveforms and confirm that the transcript truth table outputs across all three abstraction levels matched the official logic tables for every gate.

OR GATE IMPLEMENTATION

Lab_1_or_gate_bh.v | Lab_1_or_gate_df.v | Lab_1_or_gate_st.v | Lab_1_or_gate_tb.v | Compilation Report - Lab_1_OR_GATE

```
1  module Lab_1_or_gate_bh ( //Behavioral Style
2      input a,               //Declare input port 'a'
3      input b,               //Declare input port 'b'
4      output reg y           //Declares 'y' as an output port. 'reg' is required here because it is assigned inside an 'always' block.
5  );
6      always @(*) begin      //'always' block that triggers whenever any of its input signals (*) change state.
7          y = a | b;         // Performs a bitwise OR operation on inputs 'a' and 'b', assigning the result to 'y'.
8      end
9  endmodule                  //Ends the module definition
```

Figure 2.1. Behavioral Style OR GATE

Lab_1_or_gate_bh.v | Lab_1_or_gate_df.v | Lab_1_or_gate_st.v | Lab_1_or_gate_tb.v | Compila

```
1  module Lab_1_or_gate_df ( // Dataflow Style
2      input a,              // Declares 'a' as an input port
3      input b,              // Declares 'b' as an input port
4      output y              // Declares 'y' as an output port (defaults to a 'wire' type)
5  );
6
7      assign y = a | b;     // Continuously drives the output 'y' with the logical OR of 'a' and 'b'.
8  endmodule                 // Ends the module definition
```

Figure 2.2. Dataflow Style OR GATE

Lab_1_or_gate_bh.v | Lab_1_or_gate_df.v | Lab_1_or_gate_st.v

```
1  module Lab_1_or_gate_st ( // Structural Style
2      input a,              // Declares 'a' as an input port
3      input b,              // Declares 'b' as an input port
4      output y              // Declares 'y' as an output port
5  );
6
7      or or1 (y, a, b);     // Pre-defined OR gate into the module.
8
9  endmodule                 // Ends the module definition
```

Figure 2.3. Structural Style OR GATE

Lab_1_or_gate_bh.v | Lab_1_or_gate_df.v | Lab_1_or_gate_st.v | Lab_1_or_gate_tb.v* | Compilation Report -

```
1  // OR GATE Testbench
2  `timescale 1ns / 1ps // Sets the reference time unit to 1 ns and precision to 1 ps
3
4  module Lab_1_or_gate_tb;
5
6      // 1. Signal Declarations
7      reg a, b;                    // 'reg' types hold procedural values driven into the DUT inputs
8      wire y_df, y_bh, y_st;  // 'wire' types receive output signals driven by each DUT
9
10     // 2. Instantiate all three modeling styles (Design Under Test)
11     Lab_1_or_gate_df dut_df (.a(a), .b(b), .y(y_df));
12     Lab_1_or_gate_bh dut_bh (.a(a), .b(b), .y(y_bh));
13     Lab_1_or_gate_st dut_st (.a(a), .b(b), .y(y_st));
14
15     // 3. Stimulus and Monitoring Block
16     initial begin
17         // $display prints the header text once at the start of simulation in the Transcript console
18         $display("Time\tA B | Y_df Y_bh Y_st");
19
20         // $monitor dynamically prints a line whenever any listed variable (a, b, y_df, etc.) changes state
21         $monitor("%0t\t%b %b |   %b   %b    %b", $time, a, b, y_df, y_bh, y_st);
22
23         // Apply truth table test vectors sequentially with 10 ns delays
24         a = 0; b = 0; #10;
25         a = 0; b = 1; #10;
26         a = 1; b = 0; #10;
27         a = 1; b = 1; #10;
28
29         // Explicitly terminates the simulation run
30         $finish;
31     end
32
33 endmodule
```

Figure 2.4. OR GATE Testbench

NOT GATE IMPLEMENTATION

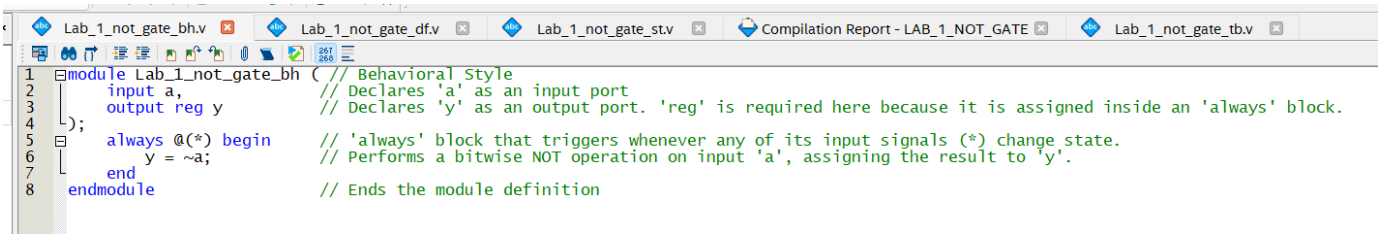


Figure 2.5. NOT GATE Behavioral Style

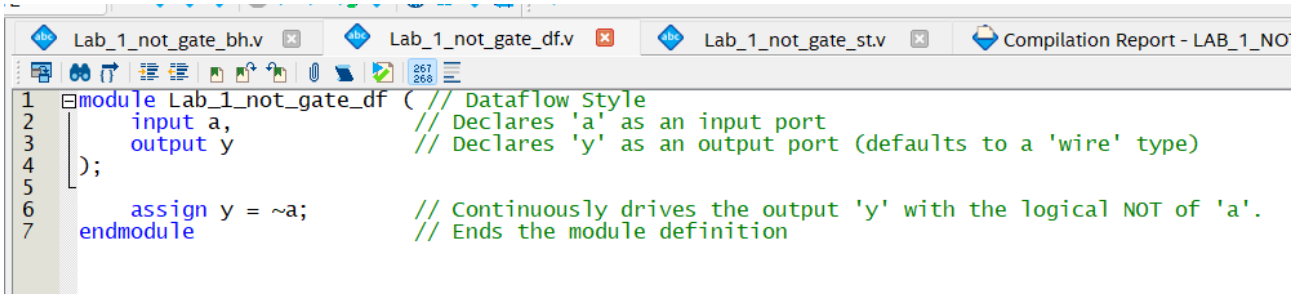


Figure 2.6. NOT GATE Dataflow Style

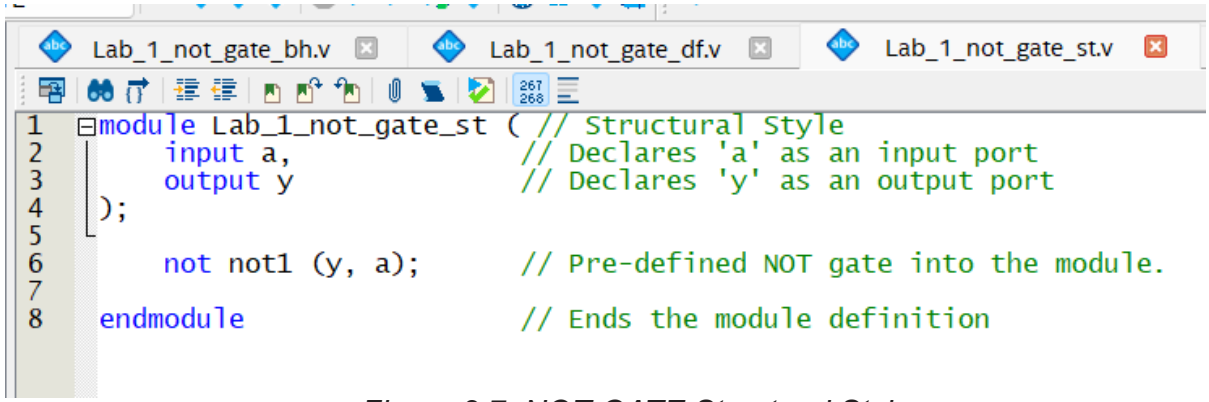


Figure 2.7. NOT GATE Structural Style

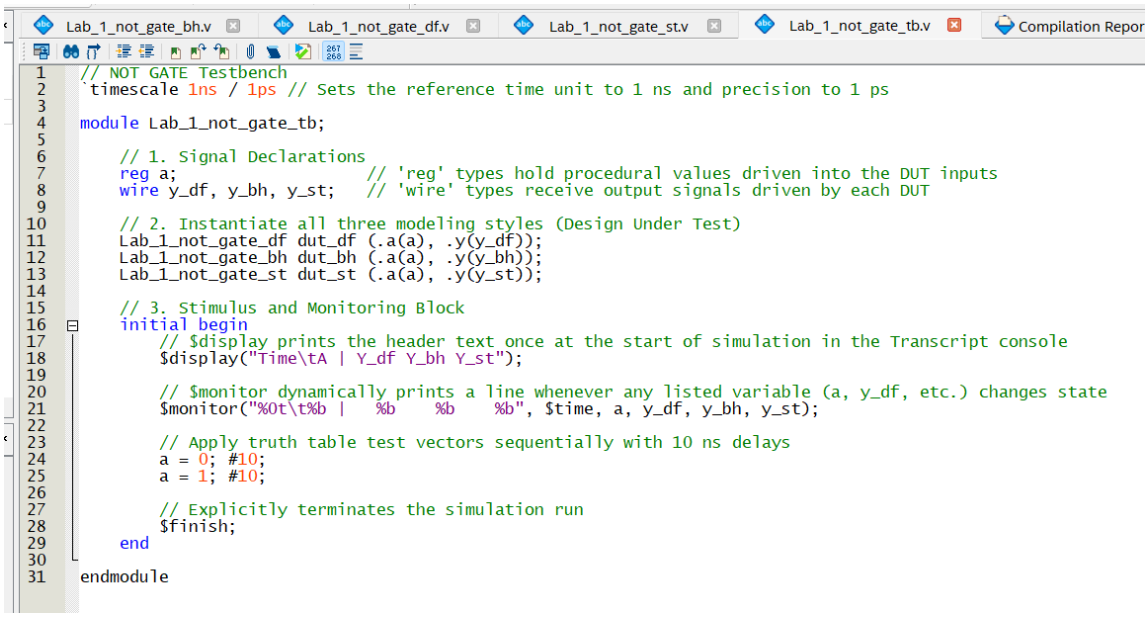


Figure 2.8. NOT GATE Testbench

NAND GATE IMPLEMENTATION

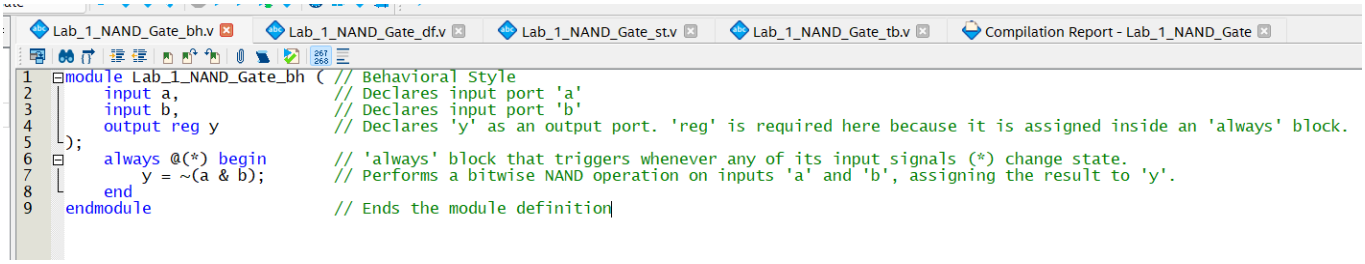


Figure 2.9. NAND GATE Behavioral Style

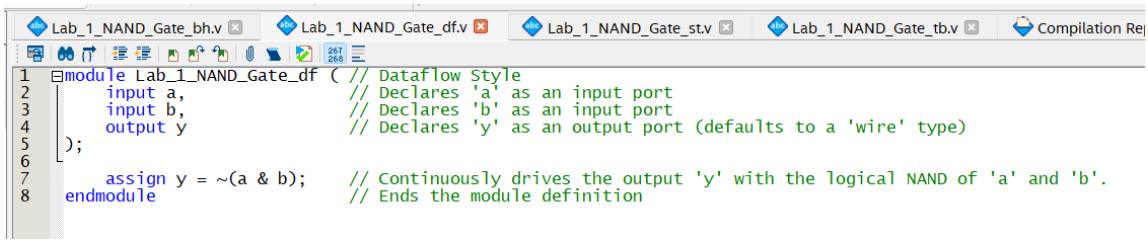


Figure 2.10. NAND GATE Dataflow Style

Lab_1_NAND_Gate_bh.v | Lab_1_NAND_Gate_df.v | Lab_1_NAND_Gate_st.v

```
module Lab_1_NAND_Gate_st ( // Structural Style
    input a,                // Declares 'a' as an input port
    input b,                // Declares 'b' as an input port
    output y                // Declares 'y' as an output port
);

    nand nand1 (y, a, b);   // Pre-defined NAND gate into the module

endmodule                   // Ends the module definition
```

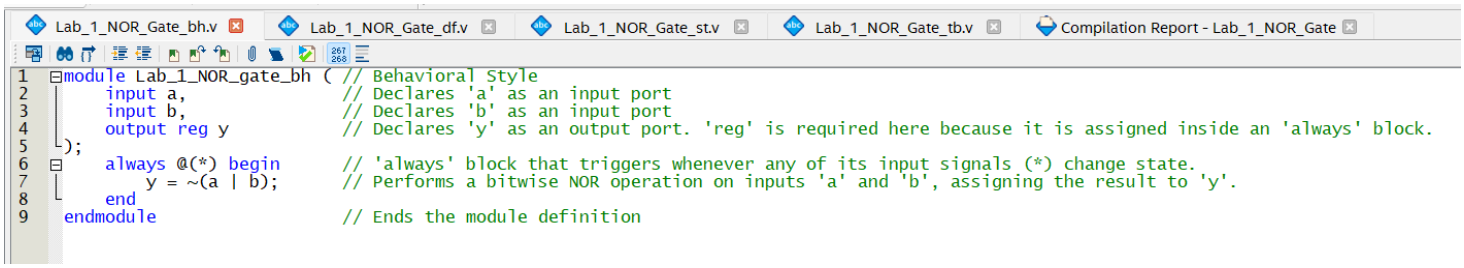
Figure 2.11. NAND GATE Structural Style

Lab_1_NAND_Gate_bh.v | Lab_1_NAND_Gate_df.v | Lab_1_NAND_Gate_st.v | Lab_1_NAND_Gate_tb.v | Compilation Repo

```
1  // NAND GATE Testbench
2  `timescale 1ns / 1ps // Sets the reference time unit to 1 ns and precision to 1 ps
3
4  module Lab_1_NAND_Gate_tb;
5
6      // 1. Signal Declarations
7      reg a, b;               // 'reg' types hold procedural values driven into the DUT inputs
8      wire y_df, y_bh, y_st;  // 'wire' types receive output signals driven by each DUT
9
10     // 2. Instantiate all three modeling styles (Design Under Test)
11     Lab_1_NAND_Gate_df dut_df (.a(a), .b(b), .y(y_df));
12     Lab_1_NAND_Gate_bh dut_bh (.a(a), .b(b), .y(y_bh));
13     Lab_1_NAND_Gate_st dut_st (.a(a), .b(b), .y(y_st));
14
15     // 3. Stimulus and Monitoring Block
16     initial begin
17         // $display prints the header text once at the start of simulation in the Transcript console
18         $display("Time\tA B | Y_df Y_bh Y_st");
19
20         // $monitor dynamically prints a line whenever any listed variable (a, b, y_df, etc.) changes state
21         $monitor("%0t\t%b %b |   %b    %b    %b", $time, a, b, y_df, y_bh, y_st);
22
23         // Apply truth table test vectors sequentially with 10 ns delays
24         a = 0; b = 0; #10;
25         a = 0; b = 1; #10;
26         a = 1; b = 0; #10;
27         a = 1; b = 1; #10;
28
29         // Explicitly terminates the simulation run
30         $finish;
31     end
32
33 endmodule
```

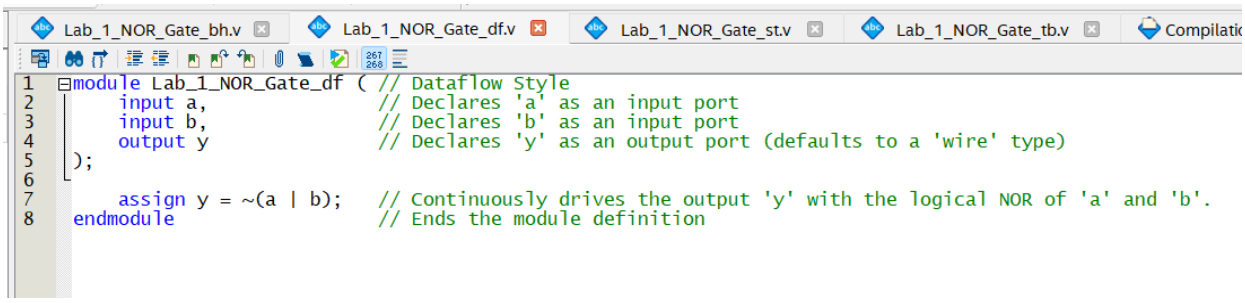
Figure 2.12. NAND GATE Test Bench

NOR GATE IMPLEMENTATION



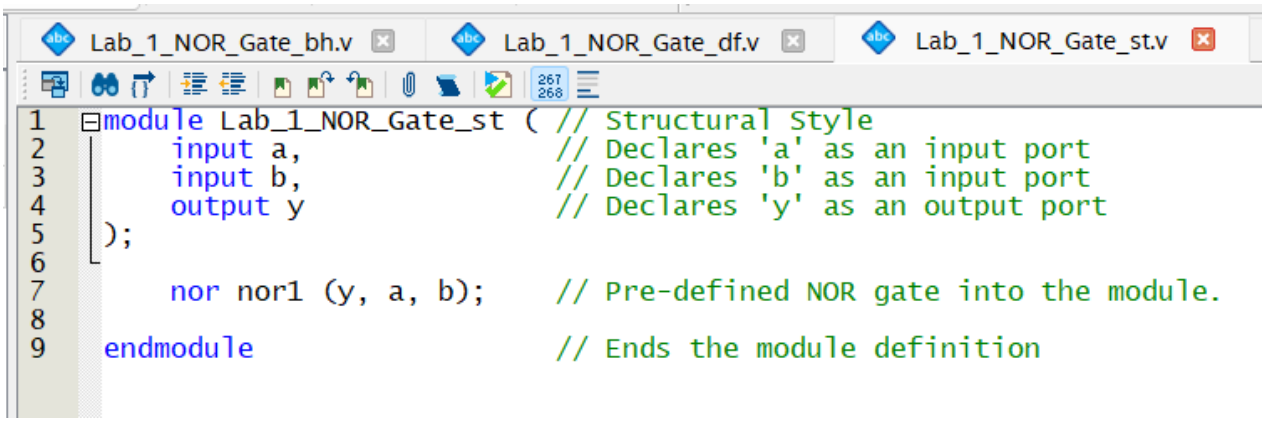
```
1  module Lab_1_NOR_gate_bh ( // Behavioral Style
2      input a,               // Declares 'a' as an input port
3      input b,               // Declares 'b' as an input port
4      output reg y           // Declares 'y' as an output port. 'reg' is required here because it is assigned inside an 'always' block.
5  );
6      always @(*) begin      // 'always' block that triggers whenever any of its input signals (*) change state.
7          y = ~(a | b);      // Performs a bitwise NOR operation on inputs 'a' and 'b', assigning the result to 'y'.
8      end
9  endmodule                  // Ends the module definition
```

Figure 2.13. NOR GATE Behavioral Style



```
1  module Lab_1_NOR_Gate_df ( // Dataflow Style
2      input a,               // Declares 'a' as an input port
3      input b,               // Declares 'b' as an input port
4      output y               // Declares 'y' as an output port (defaults to a 'wire' type)
5  );
6
7      assign y = ~(a | b);   // Continuously drives the output 'y' with the logical NOR of 'a' and 'b'.
8  endmodule                  // Ends the module definition
```

Figure 2.14. NOR GATE Dataflow Style



```
1  module Lab_1_NOR_Gate_st ( // Structural Style
2      input a,               // Declares 'a' as an input port
3      input b,               // Declares 'b' as an input port
4      output y               // Declares 'y' as an output port
5  );
6
7      nor nor1 (y, a, b);    // Pre-defined NOR gate into the module.
8
9  endmodule                  // Ends the module definition
```

Figure 2.15. NOR GATE Structural Style

Lab_1_NOR_Gate_bh.v | Lab_1_NOR_Gate_df.v | Lab_1_NOR_Gate_st.v | Lab_1_NOR_Gate_tb.v | Compilation Rep

```
// NOR GATE Testbench
`timescale 1ns / 1ps // Sets the reference time unit to 1 ns and precision to 1 ps

module Lab_1_NOR_Gate_tb;

    // 1. Signal Declarations
    reg a, b;               // 'reg' types hold procedural values driven into the DUT inputs
    wire y_df, y_bh, y_st;  // 'wire' types receive output signals driven by each DUT

    // 2. Instantiate all three modeling styles (Design Under Test)
    Lab_1_NOR_Gate_df dut_df (.a(a), .b(b), .y(y_df));
    Lab_1_NOR_gate_bh dut_bh (.a(a), .b(b), .y(y_bh));
    Lab_1_NOR_Gate_st dut_st (.a(a), .b(b), .y(y_st));

    // 3. Stimulus and Monitoring Block
    initial begin
        // $display prints the header text once at the start of simulation in the Transcript console
        $display("Time\tA B | Y_df Y_bh Y_st");

        // $monitor dynamically prints a line whenever any listed variable (a, b, y_df, etc.) changes state
        $monitor("%0t\t%b %b |   %b    %b    %b", $time, a, b, y_df, y_bh, y_st);

        // Apply truth table test vectors sequentially with 10 ns delays
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        // Explicitly terminates the simulation run
        $finish;
    end

endmodule
```

Figure 2.16. NOR GATE Testbench

XOR GATE IMPLEMENTATION

Lab_1_XOR_Gate_bh.v | Lab_1_XOR_Gate_df.v | Lab_1_XOR_Gate_st.v | Lab_1_XOR_Gate_tb.v | Compilation Report - Lab_1_XOR_Gate

```
module Lab_1_XOR_Gate_bh ( // Behavioral Style
    input a,               // Declares 'a' as an input port
    input b,               // Declares 'b' as an input port
    output reg y           // Declares 'y' as an output port. 'reg' is required here because it is assigned inside an 'always' block.
);
    always @(*) begin      // 'always' block that triggers whenever any of its input signals (*) change state.
        y = a ^ b;         // Performs a bitwise XOR operation on inputs 'a' and 'b', assigning the result to 'y'.
    end
endmodule                  // Ends the module definition
```

Figure 2.17. XOR GATE Behavioral Style

Lab_1_XOR_Gate_bh.v | Lab_1_XOR_Gate_df.v | Lab_1_XOR_Gate_st.v | Lab_1_XOR_Gate_tb.v | Compilatio

```
module Lab_1_XOR_Gate_df ( // Dataflow Style
    input a,               // Declares 'a' as an input port
    input b,               // Declares 'b' as an input port
    output y               // Declares 'y' as an output port (defaults to a 'wire' type)
);

    assign y = a ^ b;      // Continuously drives the output 'y' with the logical XOR of 'a' and 'b'.
endmodule                  // Ends the module definition
```

Figure 2.18. XOR GATE Dataflow Style

Lab_1_XOR_Gate_bh.v | Lab_1_XOR_Gate_df.v | Lab_1_XOR_Gate_st.v

```
1  module Lab_1_XOR_Gate_st ( // Structural Style
2      input a,               // Declares 'a' as an input port
3      input b,               // Declares 'b' as an input port
4      output y               // Declares 'y' as an output port
5  );
6
7      xor xor1 (y, a, b);    // Pre-defined XOR gate into the module.
8
9  endmodule                  // Ends the module definition
```

Figure 2.19. XOR GATE Structural Style

Lab_1_XOR_Gate_bh.v | Lab_1_XOR_Gate_df.v | Lab_1_XOR_Gate_st.v | Lab_1_XOR_Gate_tb.v | Compilation Report

```
1  // XOR GATE Testbench
2  `timescale 1ns / 1ps // Sets the reference time unit to 1 ns and precision to 1 ps
3
4  module Lab_1_XOR_Gate_tb;
5
6      // 1. Signal Declarations
7      reg a, b;               // 'reg' types hold procedural values driven into the DUT inputs
8      wire y_df, y_bh, y_st;  // 'wire' types receive output signals driven by each DUT
9
10     // 2. Instantiate all three modeling styles (Design Under Test)
11     Lab_1_XOR_Gate_df dut_df (.a(a), .b(b), .y(y_df));
12     Lab_1_XOR_Gate_bh dut_bh (.a(a), .b(b), .y(y_bh));
13     Lab_1_XOR_Gate_st dut_st (.a(a), .b(b), .y(y_st));
14
15     // 3. Stimulus and Monitoring Block
16     initial begin
17         // $display prints the header text once at the start of simulation in the Transcript console
18         $display("Time\tA B | Y_df Y_bh Y_st");
19
20         // $monitor dynamically prints a line whenever any listed variable (a, b, y_df, etc.) changes state
21         $monitor("%0t\t%b %b |   %b    %b    %b", $time, a, b, y_df, y_bh, y_st);
22
23         // Apply truth table test vectors sequentially with 10 ns delays
24         a = 0; b = 0; #10;
25         a = 0; b = 1; #10;
26         a = 1; b = 0; #10;
27         a = 1; b = 1; #10;
28
29         // Explicitly terminates the simulation run
30         $finish;
31     end
32
33 endmodule
```

Figure 2.20. XOR GATE Testbench

XNOR IMPLEMENTATION

Lab_1_XNOR_Gate_bh.v | Lab_1_XNOR_Gate_df.v | Lab_1_XNOR_Gate_st.v | Lab_1_XNOR_Gate_tb.v | Compilation Report - Lab_1_XNOR_Gate

```
1  module Lab_1_XNOR_Gate_bh ( // Behavioral Style
2      input a,                // Declares 'a' as an input port
3      input b,                // Declares 'b' as an input port
4      output reg y            // Declares 'y' as an output port. 'reg' is required here because it is assigned inside an 'always' block.
5  );
6      always @(*) begin       // 'always' block that triggers whenever any of its input signals (*) change state.
7          y = a ~^ b;         // Performs a bitwise XNOR operation on inputs 'a' and 'b', assigning the result to 'y'.
8      end
9  endmodule                   // Ends the module definition
```

Figure 2.21. XNOR GATE Behavioral Style

Lab_1_XNOR_Gate_bh.v | Lab_1_XNOR_Gate_df.v | Lab_1_XNOR_Gate_st.v | Lab_1_XNOR_Gate_tb.v | Compilation R

```
1  module Lab_1_XNOR_Gate_df ( // Dataflow Style
2      input a,                // Declares 'a' as an input port
3      input b,                // Declares 'b' as an input port
4      output y                // Declares 'y' as an output port (defaults to a 'wire' type)
5  );
6
7      assign y = a ~^ b;      // Continuously drives the output 'y' with the logical XNOR of 'a' and 'b'.
8  endmodule                   // Ends the module definition
```

Figure 2.22. XNOR GATE Dataflow Style

Lab_1_XNOR_Gate_bh.v | Lab_1_XNOR_Gate_df.v | Lab_1_XNOR_Gate_st.v

```
1  module Lab_1_XNOR_Gate_st ( // Structural Style
2      input a,                // Declares 'a' as an input port
3      input b,                // Declares 'b' as an input port
4      output y                // Declares 'y' as an output port
5  );
6
7      xnor xor1 (y, a, b);    // Pre-defined XNOR gate into the module.
8
9  endmodule                   // Ends the module definition
```

Figure 2.23. XNOR GATE Dataflow Style

Lab_1_XNOR_Gate_bh.v | Lab_1_XNOR_Gate_df.v | Lab_1_XNOR_Gate_st.v | Lab_1_XNOR_Gate_tb.v | Compilation Report

```
// XNOR GATE Testbench
`timescale 1ns / 1ps // Sets the reference time unit to 1 ns and precision to 1 ps

module Lab_1_XNOR_Gate_tb;

    // 1. Signal Declarations
    reg a, b;               // 'reg' types hold procedural values driven into the DUT inputs
    wire y_df, y_bh, y_st;  // 'wire' types receive output signals driven by each DUT

    // 2. Instantiate all three modeling styles (Design Under Test)
    Lab_1_XNOR_Gate_df dut_df (.a(a), .b(b), .y(y_df));
    Lab_1_XNOR_Gate_bh dut_bh (.a(a), .b(b), .y(y_bh));
    Lab_1_XNOR_Gate_st dut_st (.a(a), .b(b), .y(y_st));

    // 3. Stimulus and Monitoring Block
    initial begin
        // $display prints the header text once at the start of simulation in the Transcript console
        $display("Time\tA B | Y_df Y_bh Y_st");

        // $monitor dynamically prints a line whenever any listed variable (a, b, y_df, etc.) changes state
        $monitor("%0t\t%b %b |   %b    %b    %b", $time, a, b, y_df, y_bh, y_st);

        // Apply truth table test vectors sequentially with 10 ns delays
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        // Explicitly terminates the simulation run
        $finish;
    end

endmodule
```

Figure 2.24. XNOR GATE Dataflow Style

3. Experimental Results

The functional verification of all six assigned logic gates (OR, NOT, NAND, NOR, XOR, and XNOR) was successfully executed and confirmed through ModelSim-Altera RTL functional simulation. As shown in the simulation waveforms and Transcript console logs below, applying the complete sequence of test vectors produced exact truth table outputs for every gate at each 10 ns delay interval. Crucially, the concurrent output signals for dataflow (`y_df`), behavioral (`y_bh`), and structural (`y_st`) abstraction styles transitioned synchronously and remained identical across all timesteps, with no uninitialized (X) or high-impedance (Z) states observed. This demonstrates that all three HDL modeling approaches are functionally equivalent and accurately realize the intended logic operations in hardware.

OR GATE RESULTS

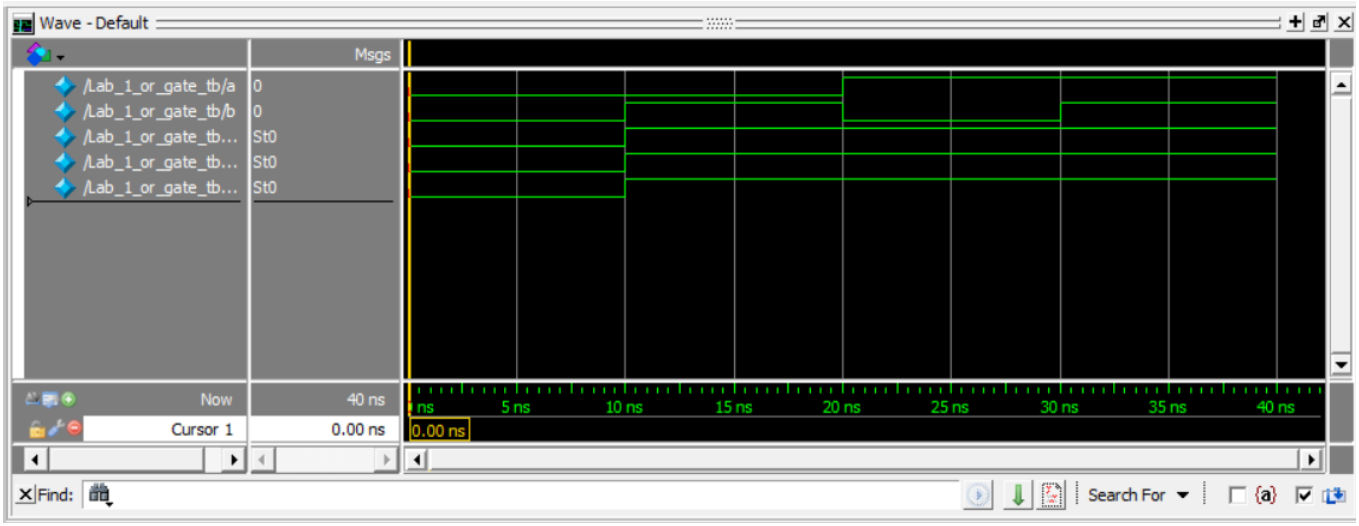


Figure 3.1. OR GATE WAVEFORM

```
# Time   A B | Y_df Y_bh Y_st
# 0      0 0 |    0    0    0
# 10000  0 1 |    1    1    1
# 20000  1 0 |    1    1    1
# 30000  1 1 |    1    1    1
```

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Figure 3.2. OR GATE Output and Truth Table Comparison

NOT GATE RESULTS

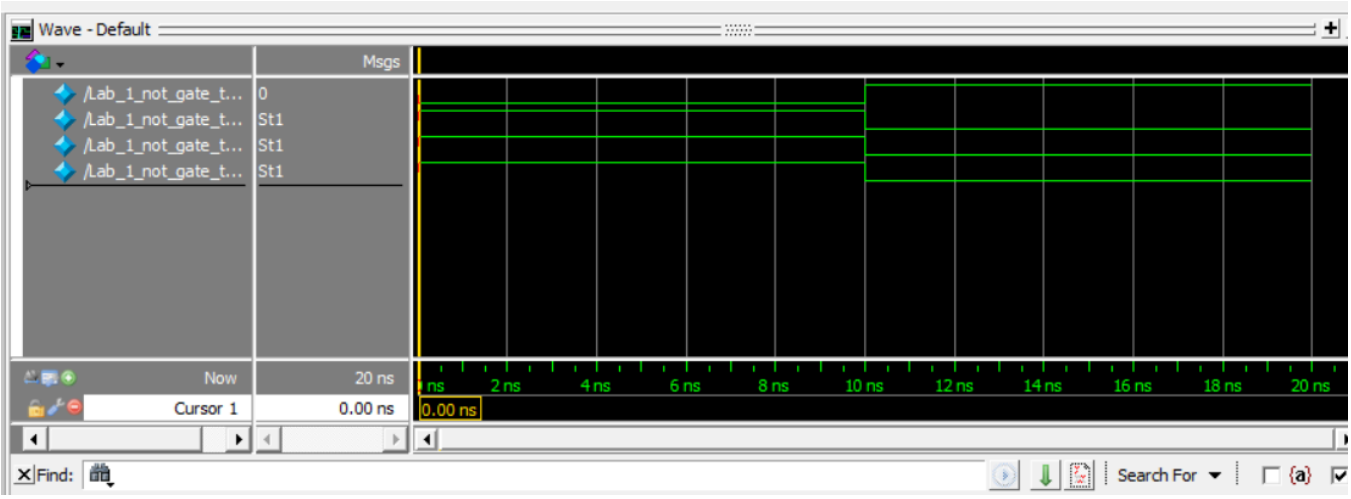


Figure 3.3. NOT GATE WAVEFORM

```
# Time   A | Y_df Y_bh Y_st
# 0      0 |   1    1    1
# 10000  1 |   0    0    0
```

p	$\neg p$
T	F
F	T

Figure 3.4. NOT GATE Output and Truth Table Comparison

NAND GATE RESULTS

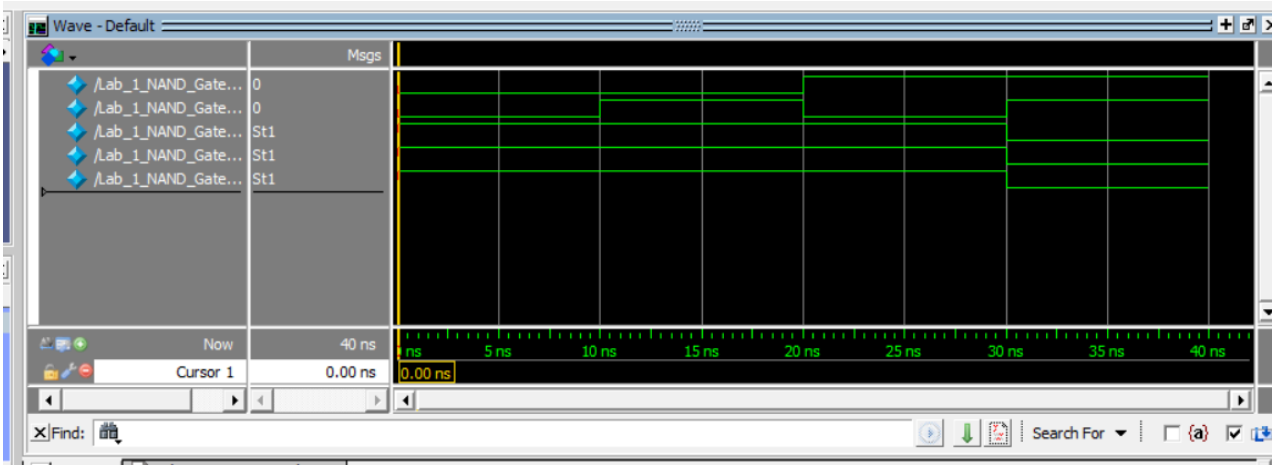


Figure 3.5. NAND GATE WAVEFORM

```
# Time   A B | Y_df Y_bh Y_st
# 0      0 0 |    1    1    1
# 10000  0 1 |    1    1    1
# 20000  1 0 |    1    1    1
# 30000  1 1 |    0    0    0
```

p	q	$\neg(p \wedge q)$
T	T	F
T	F	T
F	T	T
F	F	T

Figure 3.6. NAND GATE Output and Truth Table Comparison

NOR GATE RESULTS

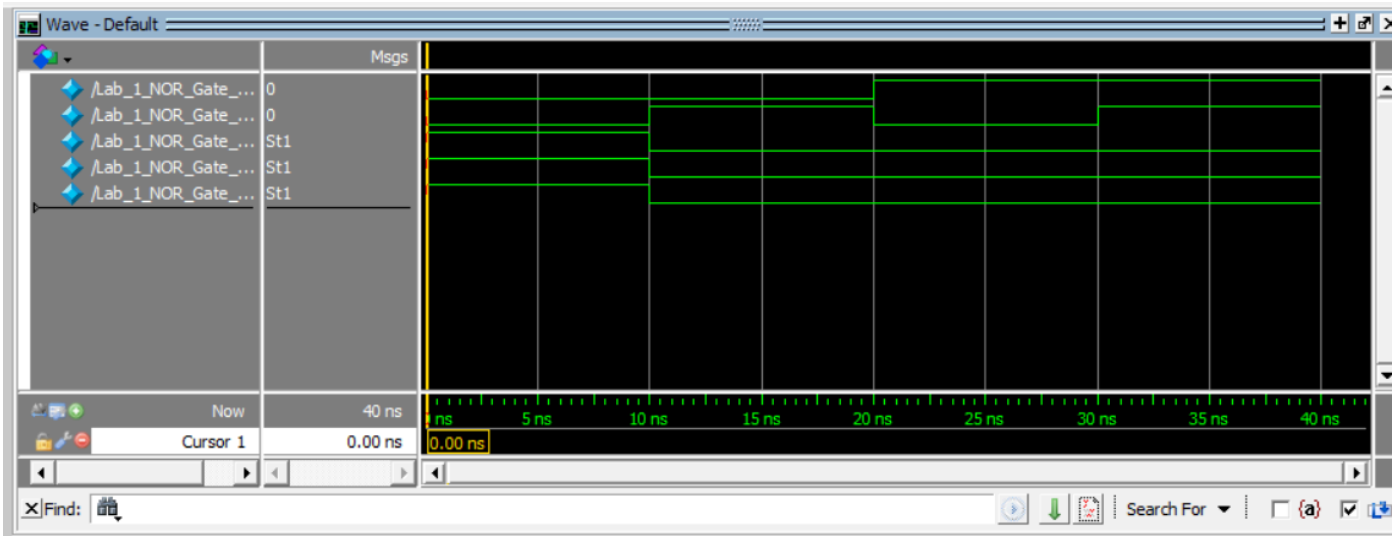


Figure 3.7. NOR GATE WAVEFORM

```
# run -all
# Time   A B | Y_df Y_bh Y_st
# 0      0 0 |    1    1    1
# 10000  0 1 |    0    0    0
# 20000  1 0 |    0    0    0
# 30000  1 1 |    0    0    0
```

p	q	$\neg(p \vee q)$
T	T	F
T	F	F
F	T	F
F	F	T

Figure 3.8. NOR GATE Output and Truth Table Comparison

XOR GATE RESULTS

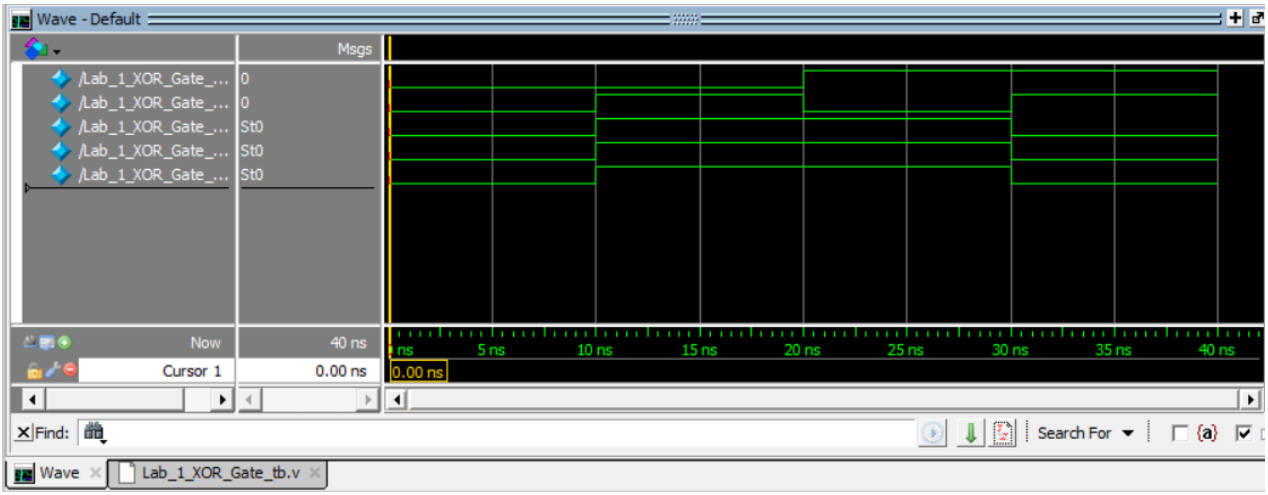


Figure 3.9. XOR GATE WAVEFORM

```
# Time   A B | Y_df Y_bh Y_st
# 0      0 0 |    0    0    0
# 10000  0 1 |    1    1    1
# 20000  1 0 |    1    1    1
# 30000  1 1 |    0    0    0
```

p	q	$(\neg p \wedge q) \vee (p \wedge \neg q)$
T	T	F
T	F	T
F	T	T
F	F	F

Figure 3.10. XOR GATE Output and Truth Table Comparison

XNOR GATE RESULTS

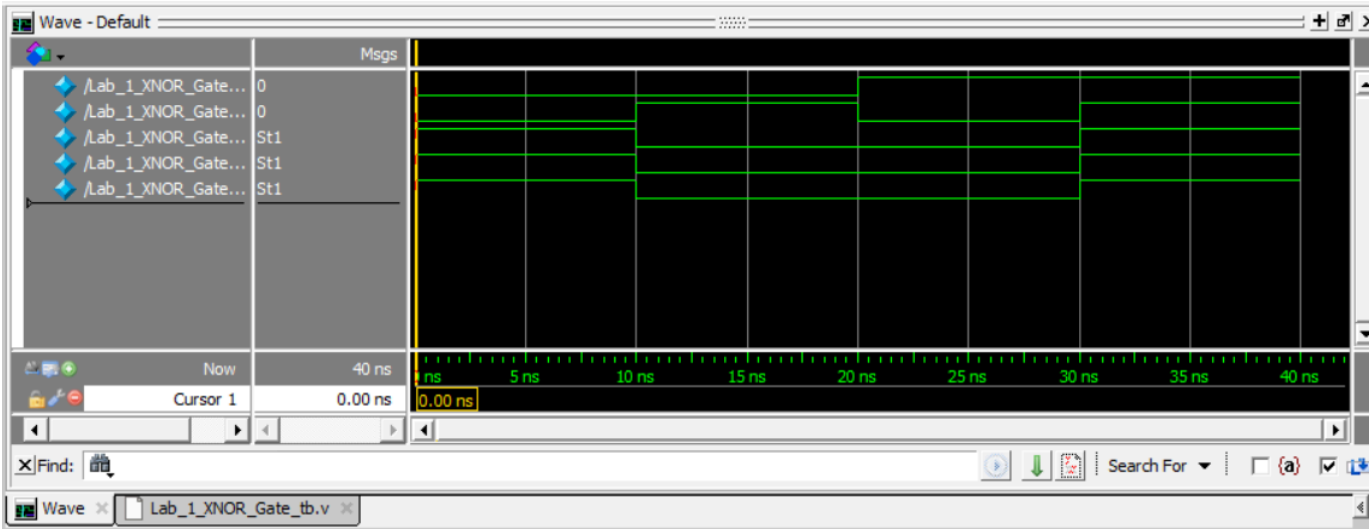


Figure 3.11. XOR GATE WAVEFORM

```
# Time   A B | Y_df Y_bh Y_st
# 0      0 0 |    1    1    1
# 10000  0 1 |    0    0    0
# 20000  1 0 |    0    0    0
# 30000  1 1 |    1    1    1
# ** Note: $finish    : C:/i
```

p	q	$(\neg p \wedge \neg q) \vee (p \wedge q)$
T	T	T
T	F	F
F	T	F
F	F	T

Figure 3.12. XOR GATE Output and Truth Table Comparison

4. Significance

This lab establishes a critical foundation in digital system design by demonstrating how fundamental logic gates serve as the basic building blocks for modern electronics. As outlined in the manual, mastering Verilog Hardware Description Language (HDL) and various abstraction levels, dataflow, behavioral, and structural, is essential because HDL simulation allows engineers to verify gate level logic and detect bugs early in software, which is significantly faster and more cost-effective than troubleshooting physical hardware. The core competencies gained from this lab, including Verilog module structuring, port declaration, and automated testbench verification, provide the necessary framework for future work in computer architecture, hardware prototyping, and complex Register-Transfer Level (RTL) design. In practical engineering applications, these principles are directly utilized to program Field-Programmable Gate Arrays (FPGAs), construct custom Integrated Circuits (ASICs) like microprocessors and memory chips, configure embedded system controllers, and design high-performance computing architectures.

5. Comments/Suggestions

At first, my main challenge was getting used to Intel Quartus Prime, finding my way around the software, and learning how to set up my design files and testbenches in the simulation settings. I also had to take time to learn the differences in Verilog syntax. For instance, I realized that behavioral outputs need to be declared as reg, which was missing in the manual's example code even though the text mentioned it. On top of that, I had to rewrite my testbench code a few times to make sure it tested all input combinations properly. Adding a quick guide for the Quartus simulation setup and fixing the example code in the manual to show output reg would make things much easier for future students. Once I understood these basic setup steps and coding rules, the rest of the lab was smooth sailing.

6. Additional Lab Specific Information

Pre-lab Questions

1. What is a truth table?

- A truth table is a mathematical table that lists every possible binary input combination for a logic circuit alongside its corresponding output state.

2. Which gates are called universal gates and why?

- NAND and NOR gates are universal gates because any Boolean logic operation or basic gate (AND, OR, NOT) can be constructed using only NAND gates or only NOR gates.

3. What is the fundamental difference between an HDL (like Verilog) and a software programming language (like C)?

- A software programming language (like C) executes instructions sequentially line-by-line on a processor, whereas a Hardware Description Language (HDL like Verilog) describes physical hardware components and concurrent signal connections that operate in parallel.

4. Define "identifier" in the context of Verilog.

- An identifier is a user-defined name given to Verilog elements, such as modules, ports, wires, variables, and instantiated components, to uniquely identify them within the source code.

5. A 2-input logic circuit has one HIGH and one LOW input, and its output is HIGH. What type of gate is it?

- It can be an OR, NAND, or XOR gate, as all three produce a HIGH output for inputs 0 and 1.

6. A logic circuit requires all its inputs to be HIGH to make the output HIGH. What type of gate is it?

- An AND gate.

Post-lab Questions

1. What is meant by "ports" in a Verilog module?

- Ports are the input and output interface signals of a Verilog module, acting like the physical pins of an integrated circuit chip to communicate with external signals or other sub-modules.

2. List the different port modes (e.g., input, output) and their purposes.

- **input**: Defines ports that receive incoming signals into the module from an external source.
- **output**: Defines ports that drive generated output signals out of the module to external connections.
- **inout**: Defines bidirectional ports that can both send and receive signals (commonly used for shared data buses).

3. Describe the different categories of operators in Verilog (e.g., bitwise, logical).

- **Bitwise Operators** (~, &, |, ^, ~^): Perform bit-by-bit logical operations on single bits or vectors.
- **Logical Operators** (!, &&, ||): Evaluate conditions across expressions and return a single bit true (1) or false (0) boolean result.
- **Arithmetic Operators** (+, -, *, /, %): Perform standard mathematical calculations on numerical values or vectors.
- **Relational Operators** (<, >, <=, >=): Compare two quantities and return a true or false result based on relative magnitude.
- **Equality Operators** (==, !=, ===, !==): Compare two values for equality, including specific case-equality variants (===, !==) that evaluate unknown (x) and high-impedance (z) states.

- **Reduction Operators** (&, ~&, |, ~|, ^, ~^): Apply a logical operation across all individual bits of a single vector to yield a 1-bit output.
- **Shift Operators** (<<, >>, <<<, >>>): Shift the bits of a signal left or right, offering both logical and arithmetic (sign-extended) shifts.
- **Concatenation and Replication Operators** ({ }, {n{ }}): Combine multiple individual signals into a single wider bus or repeat a signal multiple times.
- **Conditional Operator** (? :): Evaluates a condition to select between two expressions, directly modeling a hardware multiplexer (2 to 1 MUX).

4. What is the difference between blocking (=) and non-blocking (<=) assignment operators? (Note: We will cover this in detail for sequential logic, but research the basic difference).

- **Blocking assignment (=):** Executes sequentially line-by-line within a procedural block, blocking subsequent statements until the current assignment is evaluated and updated immediately (primarily used for combinational logic).
- **Non-blocking assignment (<=):** Evaluates all right-hand side expressions simultaneously at the current timestep and schedules updates concurrently without blocking subsequent lines (primarily used for sequential logic like flip-flops).

5. What is meant by "simulation" in the context of digital design?

- Simulation is the process of testing a digital hardware model using software tools (such as ModelSim or Icarus Verilog) by applying test inputs (stimuli) and observing output waveforms or transcript logs to verify logical correctness before implementing the design on physical hardware.